

Aula 5 — Temporizadores e razão cíclica

Quando o processador deixa de ser o relógio

Microcontroladores — DENE/UFMT

OBJETIVOS

- Justificar a existência de um temporizador a partir do erro medido de um relógio feito por espera bloqueante.
- Calcular pré-carga e divisor para um intervalo dado, a partir de T_{cy} , e dizer o papel de cada um.
- Comparar a contagem por instrução, por consulta ao indicador e por interrupção pelo erro que cada uma produz quando o processador tem outra tarefa.
- Reconhecer o temporizador como contador de eventos, cuja fonte pode ser o relógio interno ou um pino.
- Derivar período e resolução de um sinal modulado por largura de pulso a partir de PR2 e do divisor do Timer2, e escolher a frequência pela constante de tempo da carga.

1 O relógio que atrasa

Encontro	Mecanismo	O que o processador estava fazendo
0	Laço vazio contado	Executando instruções que não fazem nada
2	Contagem de ciclos	O mesmo, agora com o número certo
3	<code>__delay_us</code> , espera cega	Nada, por 5200 ciclos
4	<code>while (ADCON0bits.GO)</code>	Nada, por 60 ciclos

Quatro mecanismos, uma única técnica: **gastar instruções até que o tempo passe**. Enquanto o processador só tem uma coisa a fazer, isso basta. O preço aparece na primeira vez que ele tem duas.

O programa abaixo é um relógio de segundos nos LEDs de PORTD. A linha `TAREFA()` representa **o resto do termostato** — ler o sensor, atualizar a tela, decidir —, aqui imitado por 300 ms de espera.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define TAREFA() __delay_ms(300) /* "o resto do termostato" */

void main(void)
{
    uint8_t segundos = 0;

    LATD = 0xFF; /* LEDs apagados (ativos em 0) */
    TRISD = 0x00; /* LAT antes de TRIS: encontro 2 */

    for (;;) {
        __delay_ms(1000);
        segundos++;
        if (segundos > 59) {
            segundos = 0;
        }
        LATD = (uint8_t)~segundos;
        TAREFA();
    }
}
```

```
}
}
```

PREVISÃO

Cronometre 60 s com o relógio acima rodando. Quanto ele marca?

Escreva o número antes de rodar. Esta mesma pergunta vai ser feita a mais duas versões do programa ao longo da aula, e o placar fecha no fim.

Nenhum código desta aula traz `#pragma config`: os bits de configuração pertencem ao bootloader, e as diretivas seriam ignoradas em silêncio — **o código que veio antes do seu**.

NESTE KIT

O LED de RD0 é ativo em nível baixo (verificado na bancada), e daí o ~. O complemento supõe o mesmo para RD1 a RD7 — confirmar. Antes de escrever em LATD, as chaves que ligam os relés a PORTD precisam estar desligadas (item CH5-3/CH5-4, ainda em aberto).

O relógio atrasa porque o tempo é o código: cada volta dura 1000 ms de espera mais 300 ms de tarefa. Qualquer linha acrescentada ao laço muda o comprimento do segundo. Para que o segundo deixe de depender do programa, a contagem precisa sair do processador.

2 Um contador que anda sozinho

Um temporizador é um registrador que incrementa por conta própria, a cada pulso de uma fonte de contagem, sem executar instrução nenhuma. Quando passa do valor máximo, ele volta a zero e **levanta um indicador** — um bit num SFR.

CONCEITO

O processador não precisa acompanhar a contagem. Precisa apenas, de tempos em tempos, olhar o indicador.

O PIC18F4550 tem quatro: Timer0 e Timer1 (16 bits), Timer2 (8 bits, com comparador de período) e Timer3 (16 bits). Nesta aula interessam dois — Timer0 para contar, Timer2 porque é ele que sustenta o PWM.

2.1 A conta do pré-carregamento

A fonte de contagem interna é $F_{osc}/4$, ou seja, um incremento por ciclo de máquina: 250 ns. Um contador de 16 bits estoura em 65 536 contagens, o que dá 16,4 ms.

Para um intervalo **menor**, não se espera o estouro desde zero: parte-se de um valor mais alto.

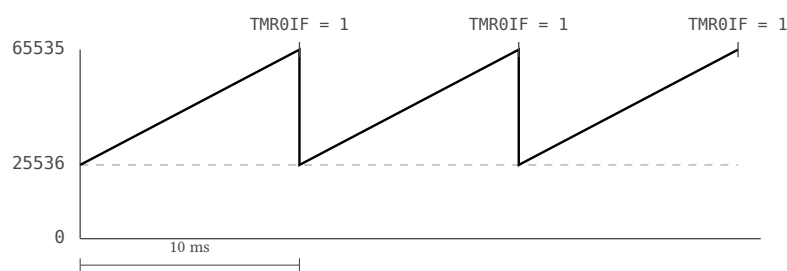
Para 10 ms, contando a 250 ns:

CONCEITO

$$N = 10 \text{ ms} / 250 \text{ ns} = 40\,000 \text{ contagens}$$

$$\text{pré-carga} = 65\,536 - 40\,000 = 25\,536 = 0x63C0$$

Escreve-se 25 536 em TMR0, e o estouro acontece 40 000 ciclos depois.



O processador não participa da subida. Ele só precisa aparecer nos instantes marcados — para zerar o indicador e recarregar 25536.

Figura 1: Um intervalo de 10 ms, três vezes. O tempo passa na diagonal; o processador só é convocado nas marcas.

2.2 E um segundo?

Um segundo são $1\text{ s} / 250\text{ ns} = 4000000$ contagens. Não cabe em dezesseis bits, e nenhuma pré-carga resolve: a pré-carga só **encurta** o caminho até o estouro.

Para alongá-lo, o Timer0 tem um **divisor** na entrada, que deixa passar um pulso a cada 2, 4, 8, até 256.

Com divisor 1:256, cada contagem vale $256 \cdot 250\text{ ns} = 64\text{ }\mu\text{s}$:

CONCEITO

$$N = 1\text{ s} / 64\text{ }\mu\text{s} = 15625\text{ contagens}$$

$$\text{pré-carga} = 65536 - 15625 = 49911 = 0xC2F7$$

Pré-carga e divisor são botões diferentes. O divisor escolhe o tamanho do passo; a pré-carga escolhe onde a contagem começa. O primeiro alonga, o segundo encurta.

A ordem das duas escritas na pré-carga não é estilo. TMR0H não é o byte alto do contador: é um **buffer**. Escrever nele não muda nada; a escrita em TMR0L é que transfere os dezesseis bits de uma vez.

ATENÇÃO

Escrever TMR0L primeiro carrega o byte baixo novo com o byte alto **anterior**, e o contador parte de um valor que ninguém pediu. O erro é intermitente, porque depende de qual era o valor anterior.

A leitura tem a simetria oposta: lê-se TMR0L primeiro, e é essa leitura que congela o byte alto no buffer.

3 O mesmo relógio, contado pelo hardware

A contagem vai para o Timer0. O laço principal passa a **consultar o indicador** TMR0IF a cada volta, e só age quando ele sobe.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define TAREFA() __delay_ms(300)
#define PRECARGA 49911u /* 65536 - 15625: 1 s a 64 us */

static void t0_recarregar(void)
{
    TMR0H = (uint8_t)(PRECARGA >> 8); /* byte alto vai para o buffer */
    TMR0L = (uint8_t)(PRECARGA & 0xFF); /* esta escrita transfere os dois */
}

void main(void)
{
```

```
uint8_t segundos = 0;

LATD = 0xFF;
TRISD = 0x00;

t0_recarregar();
T0CON = 0b10000111; /* ligado, 16 bits, relógio interno, divisor 1:256 */
INTCONbits.TMR0IF = 0;

for (;;) {
    if (INTCONbits.TMR0IF) { /* passou um segundo? */
        INTCONbits.TMR0IF = 0;
        t0_recarregar();
        segundos++;
        if (segundos > 59) {
            segundos = 0;
        }
        LATD = (uint8_t)~segundos;
    }
    TAREFA();
}
```

PREVISÃO

Mesma pergunta, mesma tarefa de 300 ms: em 60 s de cronômetro, quanto este relógio marca? Melhorou em relação ao primeiro? Ficou certo?

Melhora, mas não fica certo — e o erro tem uma forma curiosa.

O temporizador conta um segundo exato, levanta o indicador, e **continua contando**. Só que ninguém está olhando: o processador está no meio da tarefa. Ele só vê o indicador na volta seguinte do laço, e aí escreve a pré-carga **por cima** de tudo o que o contador andou nesse meio-tempo. O tempo de espera pelo olhar se perde, e se soma ao segundo.

Como a recarga acontece sempre no instante da consulta, o próximo estouro fica amarrado ao ritmo do laço. Com uma volta de 300 ms, as consultas caem em 300, 600, 900 e 1200 ms depois da recarga. O estouro acontece em 1000 ms, e quem o vê é a consulta de 1200 ms.

CONCEITO

Cada “segundo” dura exatamente **1,2 s**, e em 60 s o relógio marca 50. O erro não é aleatório: o segundo é arredondado para cima até um múltiplo da volta do laço.

São duas falhas diferentes, e vale separá-las.

NOTA

Deriva: o indicador é visto com atraso, e o atraso se soma a cada intervalo. É o que o relógio acima mostra.

Perda: se a volta do laço for **maior** que o intervalo, o indicador estoura duas vezes entre duas consultas. Como ele é um único bit, o segundo estouro some. O programa conta um evento onde houve dois.

As duas são silenciosas. Nada trava, nada reinicia — só o tempo fica errado.

3.1 A terceira forma, apresentada antes de definida

No encontro 0, main apareceu sem definição completa: uma função que nenhuma linha do seu programa chama. Quem a chama é o código que veio antes do seu, e aceitamos isso para poder começar.

A função abaixo é do mesmo tipo. Nenhuma linha do programa a chama: **quem a chama é o hardware**, no instante em que o indicador sobe, interrompendo o que quer que o processador esteja fazendo.

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define TAREFA() __delay_ms(300)
#define PRECARGA 49911u

volatile uint8_t segundos = 0;

static void t0_recarregar(void)
{
    TMR0H = (uint8_t)(PRECARGA >> 8);
    TMR0L = (uint8_t)(PRECARGA & 0xFF);
}

/* Ninguém chama esta funcao. O hardware chama. */
void __interrupt() tratar(void)
{
    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;
        t0_recarregar();
        segundos++;
        if (segundos > 59) {
            segundos = 0;
        }
    }
}

void main(void)
{
    LATD = 0xFF;
    TRISD = 0x00;

    t0_recarregar();
    T0CON = 0b10000111;
    INTCONbits.TMR0IF = 0;
    INTCONbits.TMR0IE = 1; /* o Timer0 pode interromper */
    INTCONbits.GIE = 1; /* interrupcoes ligadas no chip */

    for (;;) {
        LATD = (uint8_t)~segundos;
        TAREFA();
    }
}
```

Três coisas aceitas sem definição, a pagar no encontro 7:

ATENÇÃO

- a palavra `__interrupt()`, que marca a função que o hardware chama;
- os bits `TMR0IE` e `GIE`, que decidem **quem** pode interromper;
- a palavra `volatile` em `segundos`, e a escolha de `uint8_t` em vez de `int`.

Não tente variar nenhuma das três nesta semana. O que interessa agora é o efeito, não o mecanismo.

Com a mesma tarefa de 300 ms, o relógio marca 60 em 60 s.

Repare no que **não** melhorou: os LEDs ainda são atualizados só a cada volta do laço, então o número aparece até 300 ms depois de mudar. Mas a **contagem** não atrasa mais, porque a recarga acontece microssegundos depois do estouro, e não depois da tarefa.

NOTA

Medir e mostrar viraram coisas separadas. Só a primeira precisa ser pontual.

O bootloader ocupa o início da memória de programa, e o vetor de interrupção da aplicação é deslocado junto com ela. O projeto do curso já está configurado para isso. Se a função de interrupção nunca executar, esse deslocamento é o primeiro suspeito — é o mesmo tipo de divergência entre “onde o hardware começa” e “onde o HEX começa” do encontro 2.

NESTE KIT

Nem a interrupção zera o erro. Entre o estouro e a escrita de TMR0 ainda passam algumas dezenas de ciclos; a escrita inibe o incremento por mais dois ciclos (datasheet, seção do Timer0); e, com divisor, ela zera o que o divisor já tinha acumulado.

DIVERGÊNCIA

O resíduo é de microssegundos por segundo — da ordem da tolerância do próprio cristal. Existem formas de eliminá-lo por hardware, com um comparador que zera o temporizador sem intervenção do programa, e elas ficam fora deste curso.

3.2 O placar

Versão	Duração real de um “segundo”	Marca em 60 s
Laço com <code>__delay_ms</code>		
Consulta ao indicador		
Interrupção		

4 O contador não sabe o que é tempo

O Timer0 não mede tempo. Ele conta pulsos, e até agora os pulsos vinham do relógio interno. Mudando um bit de T0CON, eles passam a vir do pino T0CKI, que é RA4.

```
#include <xc.h>
#include <stdint.h>

void main(void)
{
    LATD = 0xFF;
    TRISD = 0x00;
    TRISAbits.TRISA4 = 1;    /* RA4 = T0CKI: entrada */

    TMR0L = 0;
    T0CON = 0b11101000;    /* ligado, 8 bits, fonte = pino T0CKI,
                             borda de subida, sem divisor */

    for (;;) {
        LATD = (uint8_t)-TMR0L;    /* o processador so mostra; quem conta e o hardware */
    }
}
```

O mesmo registrador, a mesma lógica de estouro, outra fonte. Com o relógio interno, contar pulsos é medir tempo; com um pino, é contar eventos — voltas de um eixo, pulsos de um medidor de energia, peças numa esteira.

CONCEITO

O processador não participa da contagem em nenhum dos dois casos.

PREVISÃO

Um botão está ligado a RA4, com resistor de pull-up: solto, o pino lê 1; apertado, lê 0. Você vai apertar o botão dez vezes.

- (a) O contador incrementa ao apertar ou ao soltar?
- (b) Depois de dez apertos, quanto os LEDs mostram?

Com $T0SE = 0$, a contagem acontece na borda de **subida**. Com o botão ligando o pino ao terra, a subida é o instante em que ele é **solto**.

ATENÇÃO

E dez apertos raramente dão dez. O contato mecânico repica: fecha e abre várias vezes em poucos milissegundos, e o contador — fiel — conta cada repique. O hardware não erra; o sinal é que não é o que parecia.

Esse ruído de contato é assunto do encontro 7. Por ora, basta registrar que o número medido depende do botão, e anotar quanto deu.

Verificar antes da aula: se algum botão do XM118 alcança RA4. Se não, um botão avulso na protoboard, entre RA4 e o terra, com pull-up de 10 k Ω para 5 V.

NESTE KIT

5 Gerar em vez de contar: razão cíclica

O terceiro uso do contador inverte o sentido. Até aqui ele **recebia** pulsos e o processador lia o resultado. O Timer2, acoplado a um comparador, **produz** um sinal no pino — de novo sem o processador.

O Timer2 conta até o valor de PR2 e recomeça. Um comparador liga o pino no recomeço e o desliga quando a contagem alcança o valor de CCPR1L.

CONCEITO

O resultado é uma onda quadrada de período fixo, dado por PR2, e tempo ligado ajustável, dado por CCPR1L. Mudar o tempo ligado é uma escrita num registrador; depois disso, o hardware repete a forma de onda sozinho, para sempre.

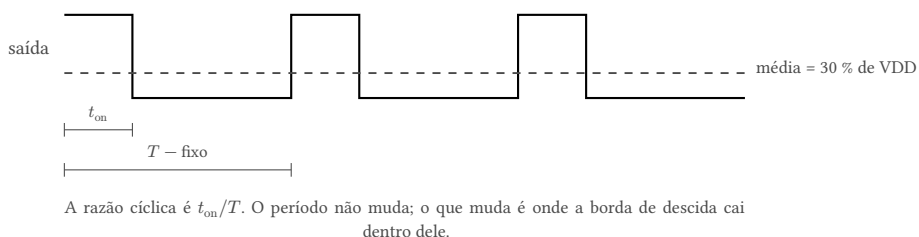


Figura 2: A razão cíclica é a única grandeza que muda. Frequência e amplitude ficam fixas.

Ligar e desligar uma carga rápido o bastante entrega a ela uma fração da energia, sem nenhum componente analógico no caminho.

A razão cíclica só equivale a uma tensão média se a carga for lenta comparada ao período.

CONCEITO

O que integra o sinal não é o microcontrolador: é a carga. A inércia mecânica da ventoinha, a inércia térmica do aquecedor e a persistência da retina fazem o trabalho. Se a carga responder mais rápido que o período, ela não vê uma média — vê exatamente o que existe, um interruptor batendo.

Carga	Constante de tempo	Frequência mínima razoável
LED	olho: ≈ 20 ms	acima de 100 Hz, ou se vê a cintilação
Ventoinha	inércia do rotor: ≈ 100 ms	algumas centenas de Hz bastam
Aquecedor	inércia térmica: dezenas de segundos	qualquer coisa acima de 1 Hz
Relé	contato mecânico: ms, e desgasta	nunca

Relé não faz PWM. Cada comutação é um evento mecânico com vida útil contada, e comutar a 1 kHz destrói o contato em minutos. O aquecedor acionado por relé é uma saída de duas posições — e a oscilação que o R9 vai medir é consequência direta disso.

ATENÇÃO

5.1 Período e resolução saem do mesmo registrador

$$T_{\text{PWM}} = (\text{PR2} + 1) \cdot 4 \cdot T_{\text{osc}} \cdot \text{divisor}$$

CONCEITO

A 16 MHz, $4 \cdot T_{\text{osc}} = 250 \text{ ns}$. Com PR2 = 255 e divisor 16:

$$T_{\text{PWM}} = 256 \cdot 250 \text{ ns} \cdot 16 = 1024 \mu\text{s} \rightarrow 976,6 \text{ Hz}$$

E a resolução da razão cíclica, em bits, é

$$\log_2(4 \cdot (\text{PR2} + 1)) = \log_2 1024 = 10 \text{ bits}$$

Período e resolução são o mesmo botão. Aumentar a frequência exige diminuir PR2, e diminuir PR2 corta bits de razão cíclica.

CONCEITO

Com PR2 = 124 e o mesmo divisor, o período cai para $500 \mu\text{s}$ — 2 kHz —, e a resolução cai para $\log_2 500 \approx 8,97$ bits. Não existe escolha que dê as duas coisas.

```
void pwm_iniciar(void)
{
    PR2      = 255;           /* 256 contagens por periodo */
    T2CON     = 0x06;         /* Timer2 ligado, divisor 1:16 */
    CCP1CON   = 0x0C;         /* modo PWM */

    LATCbits.LATC2 = 0;       /* LAT antes de TRIS: encontro 2 */
    TRISCbits.TRISC2 = 0;     /* RC2 = CCP1 */
}

/* razao: 0 a 1023. Com estes ajustes, o valor tambem e o tempo
   ligado em microssegundos. */
void pwm_razao(uint16_t razao)
{
    if (razao > 1023u) {
        razao = 1023u;
    }
    CCPR1L = (uint8_t)(razao >> 2);           /* 8 bits altos */
    CCP1CON = (uint8_t)((CCP1CON & 0xCF)
        | (uint8_t)((razao & 0x03u) << 4)); /* 2 bits baixos */
}
```

Com esses valores sai uma coincidência conveniente: o valor de dez bits da razão cíclica vale exatamente o tempo ligado **em microssegundos**, porque cada unidade dura $T_{\text{osc}} \cdot 16 = 62,5 \text{ ns} \cdot 16 = 1 \mu\text{s}$. Razão 512 é meio período, e é $512 \mu\text{s}$.

NOTA

Vale usar isso em aula e vale desconfiar depois: a coincidência morre se alguém mexer no divisor.

Os dois bits menos significativos da razão cíclica não moram em CCPR1L: moram em CCP1CON, misturados com os bits que selecionam o modo. Daí a máscara 0xCF, que preserva tudo menos esses dois.

NOTA

É a mesma situação da instrução do encontro 2 e do ADCON0 do encontro 4 — uma palavra repartida em campos de significados diferentes.

5.2 O brilho que sobe e desce

```
#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>
```

```

/* pwm_iniciar() e pwm_razao() como acima */

void main(void)
{
    uint16_t r;

    pwm_iniciar();

    for (;;) {
        for (r = 0; r < 1020u; r += 4u) {      /* sobe */
            pwm_razao(r);
            __delay_ms(10);
        }
        for (r = 1020u; r > 0u; r -= 4u) {     /* desce */
            pwm_razao(r);
            __delay_ms(10);
        }
    }
}

```

Cada rampa tem 255 passos de 10 ms: 2,55 s para subir, o mesmo para descer.

A rampa é linear na razão cíclica, mas não parece linear no brilho: o olho responde muito à variação perto do apagado e quase nada perto do máximo. O brilho dispara no começo da subida e estaciona no fim. Corrigir isso é trocar a rampa por uma tabela — mas não é assunto desta aula.

NOTA

Em RC2 está a **ventoinha**, não um LED. Para ver brilho, um LED com resistor ligado a RC2 pelo conector de expansão, com a chave da ventoinha desligada — **verificar o conector antes da aula**. Com a ventoinha ligada, a mesma rampa aparece como rotação que sobe e desce.

NESTE KIT

E aqui a aula tropeça no próprio argumento. O PWM roda sozinho, mas a **rampa** é feita com `__delay_ms` — a técnica que a primeira seção condenou. Se o laço principal tivesse outra tarefa, a rampa engasgaria.

A saída é a mesma do relógio: um intervalo de 10 ms no Timer0 (a pré-carga 0x63C0, sem divisor) e a rampa dentro da função que o hardware chama.

```

#define _XTAL_FREQ 16000000UL
#include <xc.h>
#include <stdint.h>

#define TAREFA()    __delay_ms(300)
#define PRECARGA    25536u          /* 65536 - 40000: 10 ms a 250 ns */

/* pwm_iniciar() e pwm_razao() como acima */

void __interrupt() tratar(void)
{
    static int16_t r      = 0;
    static int16_t passo = 4;

    if (INTCONbits.TMR0IF) {
        INTCONbits.TMR0IF = 0;
        TMR0H = (uint8_t)(PRECARGA >> 8);
        TMR0L = (uint8_t)(PRECARGA & 0xFF);
    }
}

```

```

    r += passo;
    if (r >= 1020) {
        passo = -4;
    } else if (r <= 0) {
        passo = 4;
    }
    pwm_razao((uint16_t)r);
}

void main(void)
{
    pwm_iniciar();

    TMR0H = (uint8_t)(PRECARGA >> 8);
    TMR0L = (uint8_t)(PRECARGA & 0xFF);
    T0CON = 0x88; /* ligado, 16 bits, relógio interno, sem divisor */
    INTCONbits.TMR0IF = 0;
    INTCONbits.TMR0IE = 1;
    INTCONbits.GIE = 1;

    for (;;) {
        TAREFA(); /* o laço principal não sabe que a rampa existe */
    }
}

```

O laço principal passa 100% do tempo bloqueado, e a rampa não engasga. Três peças trabalham sem o processador: o Timer2 gera a onda, o Timer0 marca os passos, e a interrupção faz a única coisa que exige instrução — escrever o novo valor.

Circula a afirmação de que a ventoinha “não responde à frequência de comutação” porque a inércia do rotor filtra tudo. A parte mecânica sim; o resto não. O enrolamento sofre magnetostrição na frequência de chaveamento, e uma ventoinha em PWM a 1 kHz apita a 1 kHz, com rotação perfeitamente estável.

DIVERGÊNCIA

A frequência é escolhida também por critério acústico. É o que a P5 do R6 vai medir.

O Timer2 é um só, e os dois módulos CCP dividem ele. Se ventoinha e aquecedor forem acionados por PWM, terão a mesma frequência de comutação — restrição que volta no encontro 8.

NOTA

6 Três usos, um contador

Uso	Fonte	O que o processador faz
Medir tempo	Relógio interno	Olha o indicador — ou é chamado quando ele sobe
Contar eventos	Pino T0CKI	Lê o resultado quando quiser
Gerar forma de onda	Relógio interno, via PR2 e CCPR1L	Escreve a razão cíclica e esquece

7 Previsão para o R6

PREVISÃO

P1. Com PR2 = 255 e divisor 16, qual frequência você espera medir no pino do cooler? Escreva o número antes de ligar o osciloscópio.

P2. Você vai variar a razão cíclica de 0 a 100% em passos de 10%. Em qual passo espera que o rotor comece a girar? Justifique com o que sabe sobre torque de partida.

P3. Depois de partir, você vai **reduzir** a razão cíclica. O rotor para no mesmo valor em que partiu? Diga sim ou não e por quê.

P4. Meça a frequência com o osciloscópio e compare com P1. Se divergir, o erro está em PR2, no divisor, ou na sua hipótese sobre F_{osc} ?

P5. Com razão cíclica em 50%, aproxime o ouvido do cooler. Descreva o que ouve e diga em que frequência.

Leve esta folha preenchida. A previsão é avaliada pelo raciocínio, não por acertar o número.

SEM NOTA

8 Exercícios

Exercício 5.1. Calcule a pré-carga do Timer0 para um intervalo de 2 ms, com fonte interna e sem divisor.

TAREFA

(a) Quantas contagens?

(b) Qual valor escrever em TMR0?

(c) O mesmo intervalo é possível **sem** pré-carga, usando o divisor? Justifique.

Exercício 5.2. O relógio da aula usa divisor 1:256 para contar 1 s.

TAREFA

(a) Qual é o **menor** divisor que permite contar 1 s em dezesseis bits?

(b) Calcule a pré-carga para esse divisor e para 1:128.

(c) Os três divisores (o seu, 1:128 e 1:256) dão um segundo exato? Há motivo para preferir um deles?

Exercício 5.3. O relógio por consulta ao indicador marcou 50 em 60 s com uma tarefa de 300 ms.

TAREFA

(a) Escreva a duração real de um “segundo” em função da duração L de uma volta do laço.

(b) Quanto o relógio marca com $L = 250$ ms? E com $L = 260$ ms?

(c) Um colega testa com 250 ms, vê o relógio certo e conclui que o problema foi resolvido. O que você diz a ele?

Exercício 5.4. Um programa consulta TMR0IF dentro do laço principal, e o laço contém uma atualização de tela de 1,3 ms. O intervalo do temporizador é de 1 ms.

TAREFA

(a) O que acontece?

(b) O sintoma é visível ou silencioso?

(c) Proponha duas correções.

Exercício 5.5. Você precisa de PWM a 5 kHz no cooler, a 16 MHz.

TAREFA

(a) Escolha PR2 e o divisor do Timer2.

(b) Qual resolução de razão cíclica resulta?

(c) Se o projeto exigir 10 bits de resolução, qual é a maior frequência possível?

Exercício 5.6. Explique por que a razão cíclica de 30% aplicada a um LED, a um aquecedor e a um relé produz três resultados de naturezas diferentes.

TAREFA

No encontro 6: avaliação integradora I, cobrindo os encontros 0 a 5. A interrupção aparece nela só como **efeito** — o placar do relógio —, nunca como mecanismo.

NOTA

No encontro 7: a dívida desta aula é paga. A função que o hardware chama, GIE e TMR0IE, e o porquê de volatile e de uint8_t. Junto vêm o repique que o contador

externo revelou, e um problema que esta aula não tocou: tocar uma nota no buzzer e escrever na tela ao mesmo tempo.